Metody Monte Carlo

Raport nr 2

Tadeusz Jagniewski

2025-12-25

Spis treści

1 Zadanie 1: Generowanie prób z rozkładów wielowymiarowych (5 pkt) 1
1.1 Zadania 1
1.2 Rozwiązanie 2

2 Zadanie 2: Estymacja całek metodami Monte Carlo (3 pkt) 8
2.1 Rozwiązanie 8

3 Zadanie 3: Obciążenie i metoda delta (2 pkt) 14
3.1 Rozwiązanie 14

4 Zadanie 4: Zmniejszanie wariancji w estymacji całek metodami Monte-Carlo (4 pkt) 17
4.1 Rozwiązanie 17

1 Zadanie 1: Generowanie prób z rozkładów wielowymiarowych (5 pkt)

1.1 Zadania

1. Zaproponuj i zaimplementuj w R algorytm generowania prób z 4-wymiarowego rozkładu normalnego o parametrach

$$\mu = \begin{bmatrix} 1 \\ 3 \\ -1 \\ -3 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 1 & 2 & 1 & 2 \\ 2 & 8 & 4 & 8 \\ 1 & 4 & 3 & 6 \\ 2 & 8 & 6 & 16 \end{bmatrix}.$$

2. Rozważmy następujący sympleks w $\mathbb{R}^2$:

$$S = \{(x, y) \in [0, \infty)^2 : \ x + y \leq 1\}$$

Zaproponuj i zaimplementuj w R algorytm generowania prób z rozkładu o gęstości

$$f(x, y) = 60xy^2 \mathbf{1}_S(x, y).$$

korzystając z rozkładów warunkowych. Narysuj wykres łączny (jointplot) wygenerowanej próby $((x_j, y_j))_{j=1}^n$, wraz z histogramami rozkładów brzegowych.

3. Zaproponuj i zaimplementuj w R algorytm generowania prób z rozkładu dwuwymiarowego o gęstości

$$f(x, y) = \frac{1}{C}(1 + x^2 + 2y^2)^{-4/3},$$

gdzie $C = \int_{\mathbb{R}^2}(1 + x^2 + 2y^2)^{-4/3}\mathrm{d}x\mathrm{d}y$ korzystając z metody ilorazów jednostajnych.Narysuj wykres łączny (jointplot) wygenerowanej próby $((x_j, y_j))_{j=1}^n$, wraz z histogramami rozkładów brzegowych.

Wskazówka: Należy zastosować ogólniejszą metodę ilorazów jednostajnych dla pewnego $r > 1$ (trzeba dobrać r, tak by funkcje $h_j(x, y)$ były ograniczone)

1.2 Rozwiązanie

1.2.1 Zadanie 1.1

Poniżej znajduje się program który generuje próby z d-wymiarowego rozkładu normalnego.

```
generate_mvn <- function(n, mu, Sigma) {
  d <- length(mu)

  L <- t(chol(Sigma))

  Z <- matrix(rnorm(n * d), nrow = d, ncol = n)

  X <- mu + L %*% Z

  return(t(X))
}
```

Przykładowe działanie programu dla n = 1000 oraz wartości μ i $\sum$ podanych powyżej w zadaniu. Wychodzi że:

Tabela 1: Wektor średnich empirycznych (n = 1000)

E[X1]	E[X2]	E[X3]	E[X4]
1.009	2.992	-0.99	-2.941

Tabela 2: Macierz kowariancji empirycznej (n = 1000)

	X1	X2	X3	X4
X1	0.952	1.754	0.846	1.707
X2	1.754	7.011	3.449	6.786
X3	0.846	3.449	2.698	5.363
X4	1.707	6.786	5.363	14.829

Z kolei dla n = 5000 wychodzi:

Tabela 3: Wektor średnich empirycznych (n = 5000)

E[X1]	E[X2]	E[X3]	E[X4]
0.999	2.982	-1.008	-3.046

Tabela 4: Macierz kowariancji empirycznej (n = 5000)

	X1	X2	X3	X4
X1	0.996	1.978	1.004	2.021
X2	1.978	7.820	3.935	7.853
X3	1.004	3.935	2.995	6.000
X4	2.021	7.853	6.000	16.084

Zwiększenie liczności próby z n = 1000 do n = 5000 znacząco poprawia stabilność wyników, co najlepiej widać po macierzy kowariancji, której elementy empiryczne stają się niemal identyczne z wartościami teoretycznymi $\sum$.

1.2.2 Zadanie 1.2

Aby wygenerować parę (X, Y), musimy wyznaczyć gęstość brzegową $f_X(x)$ oraz gęstość warunkową $f_{Y|X=x}(y)$.

1. Gęstość brzegowa X: Całkujemy gęstość łączną po y w granicach od 0 do 1−x:

$$f_X(x) = \int_0^{1-x} 60xy^2 dy = 20x(1-x)^3, \quad x \in [0,1]$$

Jest to gęstość rozkładu **Beta** z parametrami $\alpha = 2 \quad i \quad \beta = 4$.

2. Gęstość warunkowa $f_{Y|X=x}(y)$:

$$f_{Y|X=x}(y) = \frac{f(x,y)}{f_X(x)} = \frac{60xy^2}{20x(1-x)^3} = \frac{3y^2}{(1-x)^3}, \quad y \in [0, 1-x]$$

Całkując można łatwo obliczyć dystrybuantę warunkową, która wynosi:

$$F_{Y|X=x}(y) = \frac{y^3}{(1-x)^3}$$

Stosując metodę odwracania dystrybuanty otrzymujemy:

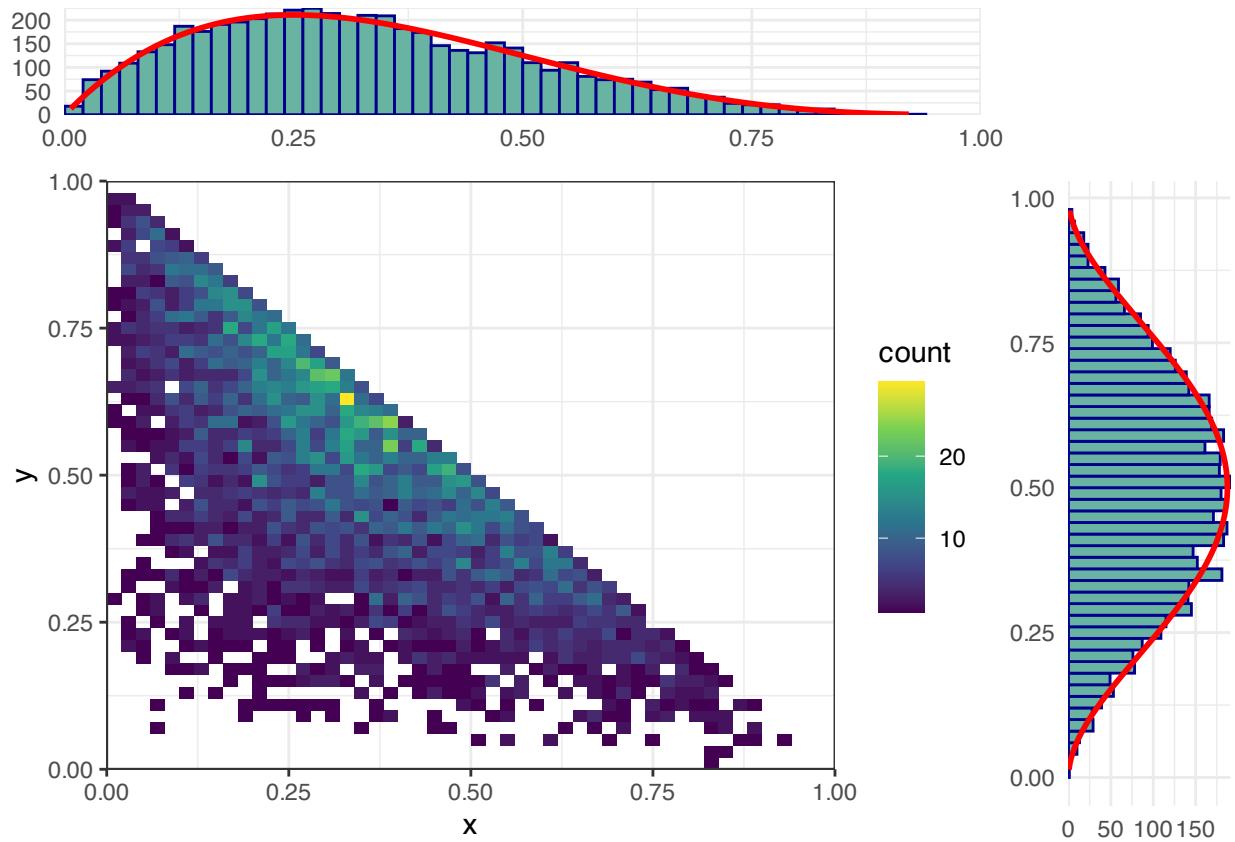
$$U = \frac{Y^3}{(1-x)^3} \implies Y = (1-x)\sqrt[3]{U}, \quad U \sim \mathcal{U}(0,1)$$

Poniżej znajduje się program, który generuje próby na podstawie powyższych obliczeń.

```
gen_simplex_60xy2 <- function(n) {
  x <- rbeta(n, shape1 = 2, shape2 = 4)

  u <- runif(n)

  y <- (1 - x) * (u^(1/3))

  return(list(x = x, y = y))
}
```

Poniżej znajduje się jointplot dla próby wielkości n = 5000, na podstawie powyższego programu

```
## Coordinate system already present. Adding new coordinate system, which will
## replace the existing one.
```



Rysunek 1: wykres łączny (jointplot) wygenerowanej próby (x,y) wraz z histogramami rozkładów brzegowych dla n = 5000

1.2.3 Zadanie 1.3

$$f(x,y) = \frac{1}{C}\,(1+x^2+2y^2)^{-4/3}, \qquad (x,y) \in \mathbb{R}^2,$$

gdzie

$$C = \int_{\mathbb{R}^2} (1+x^2+2y^2)^{-4/3}\,dx\,dy.$$

Stała normalizująca.

Podstawmy $y' = \sqrt{2}\,y$, zatem $dy = dy'/\sqrt{2}$ i

$$C = \frac{1}{\sqrt{2}} \int_{\mathbb{R}^2} (1+x^2+y'^2)^{-4/3}\,dx\,dy'.$$

Współrzędne biegunowe w $\mathbb{R}^2 : x = r\cos\theta,\ y' = r\sin\theta, dx\,dy' = r\,dr\,d\theta$:

$$C = \frac{1}{\sqrt{2}} \int_0^{2\pi} \int_0^{\infty} r(1+r^2)^{-4/3}\,dr\,d\theta = \frac{2\pi}{\sqrt{2}} \int_0^{\infty} r(1+r^2)^{-4/3}\,dr.$$

Podstawmy $t = 1+r^2 \Rightarrow dt = 2r\,dr$:

$$\int_0^{\infty} r(1+r^2)^{-4/3}\,dr = \frac{1}{2}\int_1^{\infty} t^{-4/3}\,dt = \frac{1}{2}\left[-3t^{-1/3}\right]_1^{\infty} = \frac{3}{2}.$$

Zatem

$$C = \frac{2\pi}{\sqrt{2}} \cdot \frac{3}{2} = \frac{3\pi}{\sqrt{2}}.$$

Metoda ilorazów jednostajnych w wersji ogólnej

Niech $g(x,y) = (1 + x^2 + 2y^2)^{-4/3}$. Dla $r > 0$ definiujemy zbiór:

$$A_{g,r} = \left\{ (u, v_1, v_2) \in (0,\infty) \times \mathbb{R}^2 : \ 0 < u \le \left(g(v_1/u^r,\, v_2/u^r)\right)^{\frac{1}{1+2r}} \right\}$$

Jeżeli (U, V_1, V_2) ma rozkład jednostajny na $A_{g,r}$, to

$$X = \frac{V_1}{U^r}, \qquad Y = \frac{V_2}{U^r}$$

ma gęstość proporcjonalną do g, a więc po normalizacji dokładnie f.

Dobór r

Aby móc symulować jednostajnie na $A_{g,r}$ przez odrzucanie z pudełka, potrzebujemy ograniczoności funkcji

$$h_1(x,y) = x\, g(x,y)^{\frac{r}{1+2r}}, \qquad h_2(x,y) = y\, g(x,y)^{\frac{r}{1+2r}}.$$

Dla $x^2 + 2y^2 \to \infty$ mamy $g(x,y) \asymp (x^2 + 2y^2)^{-4/3}$, ($\asymp$ oznacza, że są tego samego rzędu) więc

$$|h_j(x,y)| \asymp \|(x,y)\|\, (x^2 + 2y^2)^{-\frac{4r}{3(1+2r)}} \asymp \|(x,y)\|^{1-\frac{8r}{3(1+2r)}}.$$

Warunek ograniczoności na nieskończoności to

$$1 - \frac{8r}{3(1+2r)} \le 0 \quad \Longleftrightarrow \quad r \ge \frac{3}{2}.$$

Wybieramy więc np. $r = 2$.

Pudełko ograniczające dla r = 2:

Mamy $\sup g = 1$, więc $u \in (0,1]$. Ponadto

$$|v_1| = |xu^2| \le |x|\, g(x,y)^{\frac{2}{1+4}} = |x|(1 + x^2 + 2y^2)^{-8/15},$$

$$|v_2| = |yu^2| \le |y|(1 + x^2 + 2y^2)^{-8/15}.$$

Maksima są skończone i (np. z rachunku pochodnej wzdłuż osi) wynoszą:

$$M_x = \sup_{x,y} |x|(1 + x^2 + 2y^2)^{-8/15} = \sqrt{15} \cdot 16^{-8/15},$$

$$M_y = \sup_{x,y} |y|(1 + x^2 + 2y^2)^{-8/15} = \sqrt{\frac{15}{2}} \cdot 16^{-8/15}.$$

Zatem można odrzucać z prostokąta:

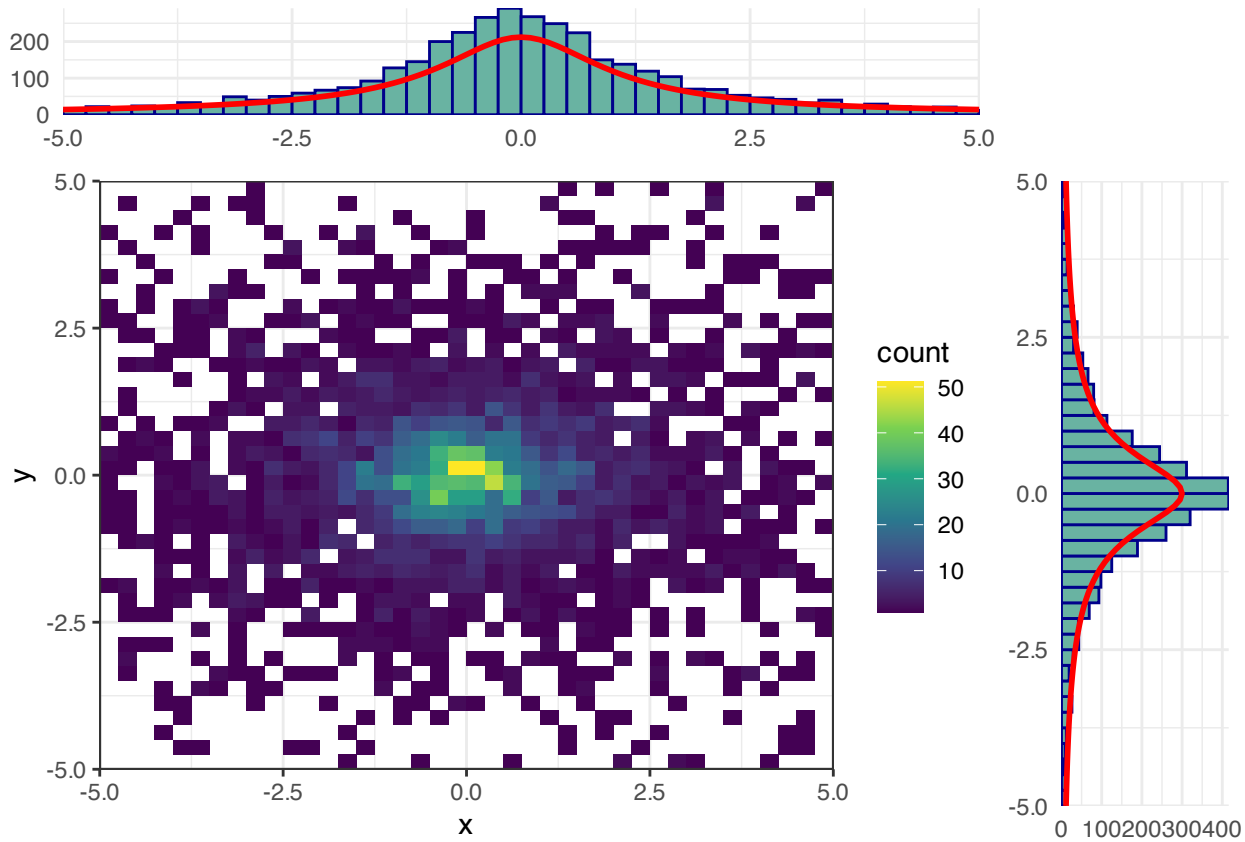
$$u \in (0,1], \quad v_1 \in [-M_x, M_x], \quad v_2 \in [-M_y, M_y],$$

akceptując tylko te (u, v_1, v_2), które należą do $A_{g,2}$.

A poniżej znajduje się ten algorytm w kodzie R:

```
gen_rou<- function(n) {
  x_res <- numeric(n)
  y_res <- numeric(n)
  r <- 2
  count <- 0

  u1_max <- sqrt(15) * (1 + 15)^(-8/15)
  u1_min <- -u1_max
  u2_max <- sqrt(7.5) * (1 + 2*7.5)^(-8/15)
  u2_min <- -u2_max

  while(count < n) {
    u0 <- runif(1, 0, 1)
    u1 <- runif(1, u1_min, u1_max)
    u2 <- runif(1, u2_min, u2_max)

    x_prop <- u1 / (u0^r)
    y_prop <- u2 / (u0^r)

    if (u0 <= (1 + x_prop^2 + 2*y_prop^2)^(-4/15)) {
      count <- count + 1
      x_res[count] <- x_prop
      y_res[count] <- y_prop
    }
  }
  return(list(x = x_res, y = y_res))
}
```

Z kolei poniżej znajduje się wykres jointplot dla próby równej n = 200, na którym widać jak rozkłdają się wartości generowane z naszego rozkładu.



Rysunek 2: Wykres łączny (jointplot) wygenerowanej próby wraz z histogramami rozkładów brzegowych dla n = 200

2 Zadanie 2: Estymacja całek metodami Monte Carlo (3 pkt)

1. Oblicz w przybliżeniu objętość elipsoidy

$$E = \{(x, y, z) \in \mathbb{R}^2 : \frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} \leq 1\}$$

   dla wybranych RÓŻNYCH parametrów $a, b, c > 0$.

2. Żuk Mandelbrota jest podzbiorem płaszczyzny zespolonej, do którego należą punkty $p \in \mathbb{C}$, dla których ciąg

$$z_n = \begin{cases} z_0 = 0 \\ z_n = z_{n-1}^2 + p, \ n > 1 \end{cases}$$

   nie zbiega do nieskończoności. Oblicz przybliżoną powierzchnię zbioru Mandelbrota (rozumianego jako podzbiór płaszczyzny $\mathbb{R}^2$).

3. Oblicz przybliżoną wartość całki

$$\int_0^{2\pi} x \sin\left[(1/\cos\left(\ln\left(x+1\right)\right))^2\right] \mathrm{d}x.$$

Dla każdego z zaimplementowanych estymatorów, stwórz wykres zmienności wartości estymatora od liczności próby i zaznacz na nim asymptotyczne przedziały ufności.

2.1 Rozwiązanie

2.1.1 Zadanie 2.1

Objętość elipsoidy o półosiach a,b,c wyraża się ogólnym wzorem:

$$V = \frac{4}{3}\pi abc$$

Algorytm estymacji objętości

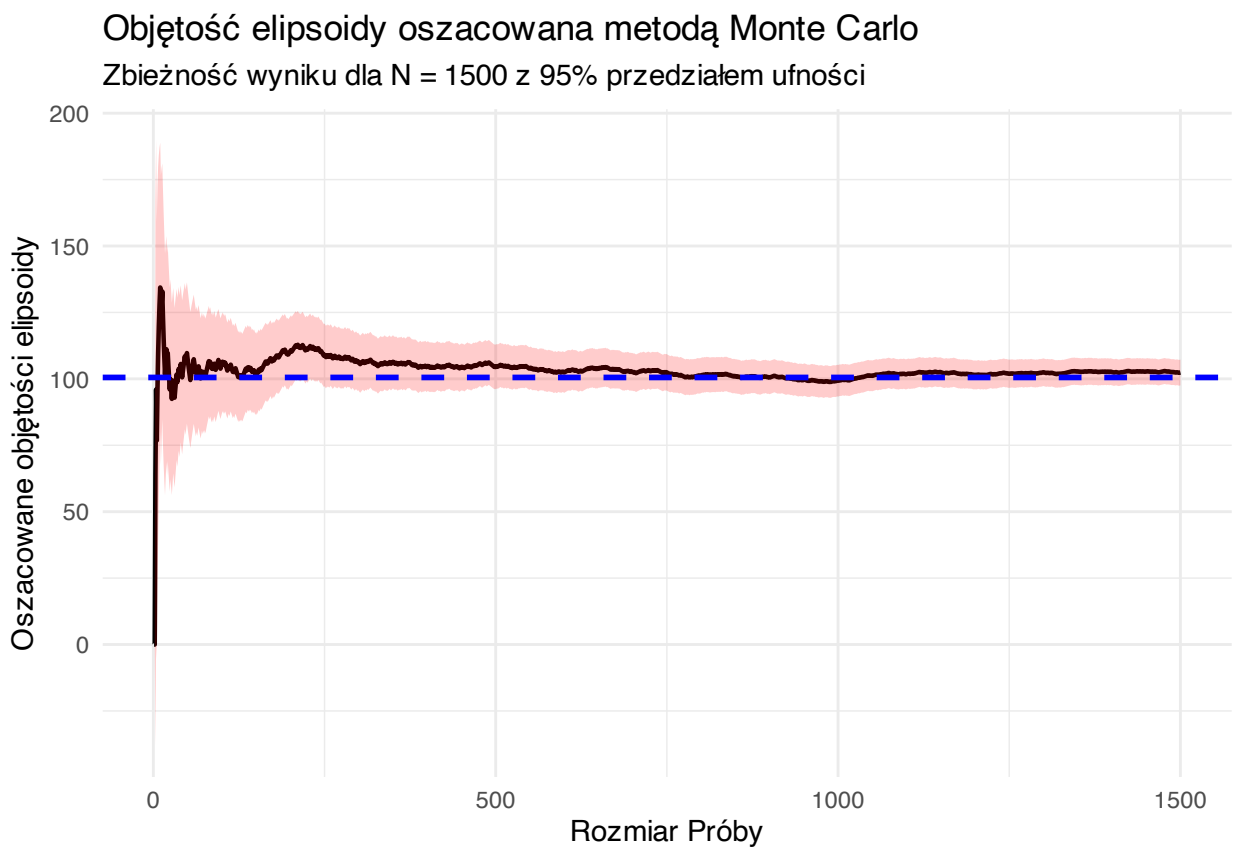
- Zamykamy elipsoidę w prostopadłościanie (pudle) ograniczonym przez [−a,a]×[−b,b]×[−c,c].
- Objętość tego pudła wynosi $V_{box} = (2a) \cdot (2b) \cdot (2c) = 8abc$.
- Losujemy N punktów z rozkładu jednostajnego wewnątrz tego pudła.
- Sprawdzamy, ile punktów wpada do elipsoidy
- Estymator objętości to: $\hat{V} = V_{box} \frac{N_{trafionych}}{N}$

A poniżej znajduje się program który to oblicza:

```
symulacja_elipsoida <- function(N, a, b, c) {

  vol_box <- (2 * a) * (2 * b) * (2 * c)

  x <- runif(N, min = -a, max = a)
  y <- runif(N, min = -b, max = b)
  z <- runif(N, min = -c, max = c)

  inside_ellipsoid <- (x/a)^2 + (y/b)^2 + (z/c)^2 <= 1

  current_hits <- cumsum(inside_ellipsoid)
```

```r
  n_samples <- 1:N

  p_hat <- current_hits / n_samples

  estimations <- p_hat * vol_box

  se_vec <- sqrt(p_hat * (1 - p_hat) / n_samples) * vol_box

  z_score <- 1.96
  ci_dolne <- estimations - (z_score * se_vec)
  ci_gorne <- estimations + (z_score * se_vec)

  return(data.frame(
    sample_size = n_samples,
    estimation = estimations,
    ci_dolne = ci_dolne,
    ci_gorne = ci_gorne
  ))
}
```

A poniżej znajduje się wykres przedstawiajacy działanie powyższego programu dla przykładowych wartości (a,b,c) = (2,3,4) i rosnącego rozmiaru próby.



Rysunek 3: Wykres objętości elipsoidy szacowanej metodą Monte Carlo z zaznaczonymi przedziałami ufności dla rosnącego rozmiaru próby

2.1.2 Zadanie 2.2

Zbiór Mandelbrota definiuje się jako zbiór parametrów c $\in \mathbb{C}$, , dla których ciąg z_n, dany rekurencją:

$$z_0 = 0 \ , \ z_{n+1} = z_n^2 + c$$

pozostaje ograniczony. W obliczeniach korzystamy z równoważnego warunku „escape radius": jeżeli dla pewnego n zachodzi $|z_n| > 2$, to ciąg jest rozbieżny, więc $c \notin M$. W konsekwencji, aby sprawdzić przynależność punktu c do M, iterujemy rekurencję do chwili, gdy $|z_n| > 2$ (wtedy przerywamy obliczenia dla tego punktu), albo do osiągnięcia limitu iteracji *max_iter* (wtedy przerywamy obliczenia dla tego punktu).

Pole zbioru szacujemy metodą Monte Carlo: losujemy N punktów jednostajnie w prostokącie obejmującym M tj. [-2,5;1,5] x [-1,25;1,25] o polu $P_{box} = 10$, zliczamy odsetek punktów, które nie uciekły, i mnożymy przez pole:

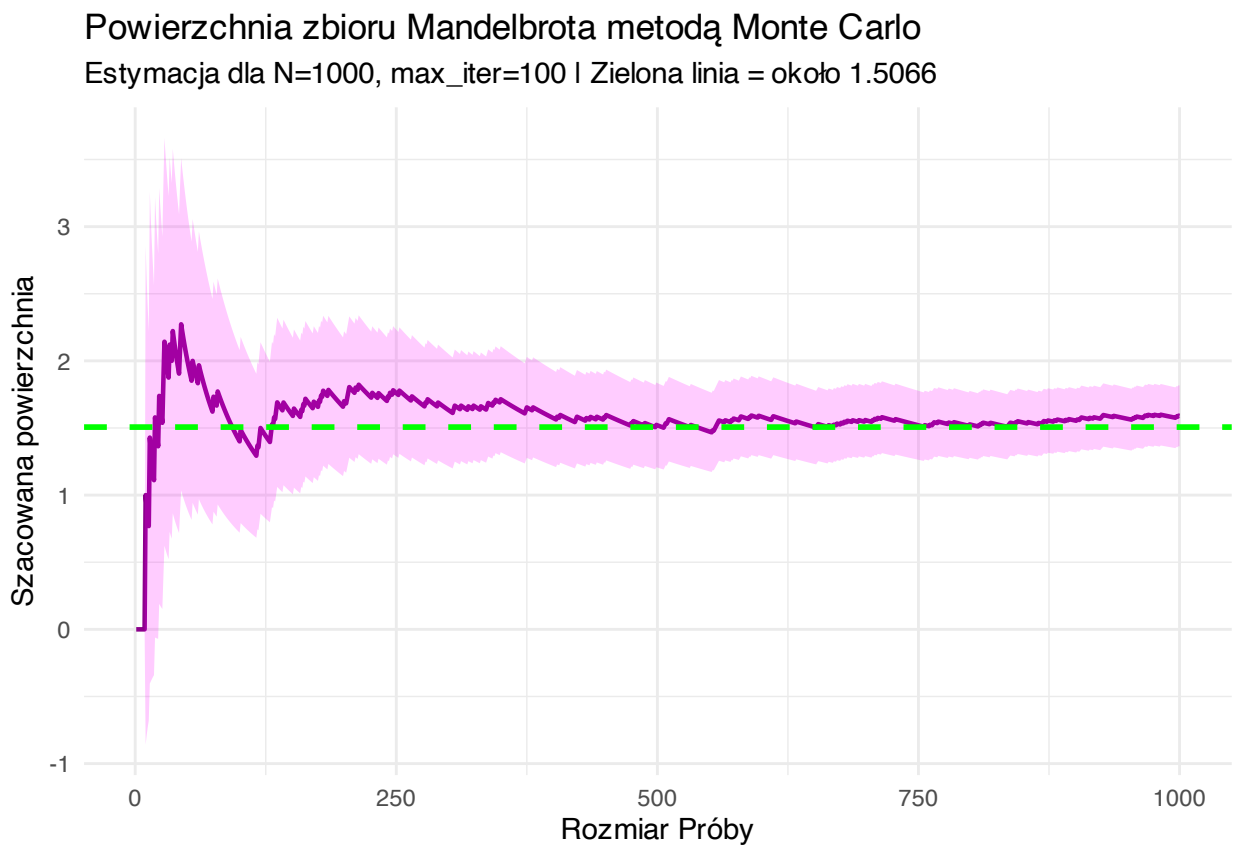
$$\widehat{P(M)} = \frac{H}{N} \cdot P_{box}$$

gdzie H oznacza liczbę trafień.

```r
symulacja_mandelbrot <- function(N, max_iter = 100) {

  x_min <- -2.5; x_max <-  1.5
  y_min <- -1.25; y_max <- 1.25

  vol_box <- (x_max - x_min) * (y_max - y_min)

  x <- runif(N, min = x_min, max = x_max)
  y <- runif(N, min = y_min, max = y_max)
  cplx <- complex(real = x, imaginary = y)

  z <- rep(0 + 0i, N)

  inside_mask <- rep(TRUE, N)

  for (i in 1:max_iter) {
    active <- which(inside_mask)
    if (length(active) == 0) break

    z[active] <- z[active]^2 + cplx[active]

    escaped_now <- (Re(z[active])^2 + Im(z[active])^2) > 4

    inside_mask[active[escaped_now]] <- FALSE
  }

  current_hits <- cumsum(inside_mask)
  n_samples <- 1:N

  p_hat <- current_hits / n_samples
  estimations <- p_hat * vol_box

  se_vec <- sqrt(p_hat * (1 - p_hat) / n_samples) * vol_box
  z_score <- 1.96

  ci_dolne <- estimations - z_score * se_vec
```

```
  ci_gorne <- estimations + z_score * se_vec

  data.frame(
    sample_size = n_samples,
    estimation  = estimations,
    ci_dolne    = ci_dolne,
    ci_gorne    = ci_gorne
  )
}
```

Poniżej znajduje się wykres z zastosowaniem programu dla N = 1000 i max_iter = 100 z przedziałem ufności 95%.



Rysunek 4: Wykres Pola Żuka Mandelbrota szacowanego metodą Monte Carlo z zaznaczonymi przedziałami ufnościdla rosnącego rozmiaru próby

2.1.3 Zadanie 2.3

Trzeba obliczyć przybliżoną wartość całki

$$\int_0^{2\pi} x \sin\left[(1/\cos\left(\ln\left(x+1\right)\right))^2\right] \mathrm{d}x.$$

Aby to zrobić użyję metody Importance Sampling, skoro amplituda funkcji rośnie liniowo jak x, użyjmy gęstości prawdopodobieństwa proporcjonalnej do x:

$$g(x) = \frac{x}{2\pi^2}, \ dla \ x \in [0, 2\pi]$$

Aby wylosować liczby z takiego rozkładu, używamy metody odwracania dystrybuanty: Jeśli U $\sim$ U(0,1), to X $= 2\pi\sqrt{U}$ będzie miało rozkład liniowy. Niech $w(x) = \frac{f(x)}{g(x)}$ i całkę przybliżamy metodą monte carlo:

$$\hat{I}_n = \frac{1}{n}\sum_{i=1}^{n} w(X_i)$$

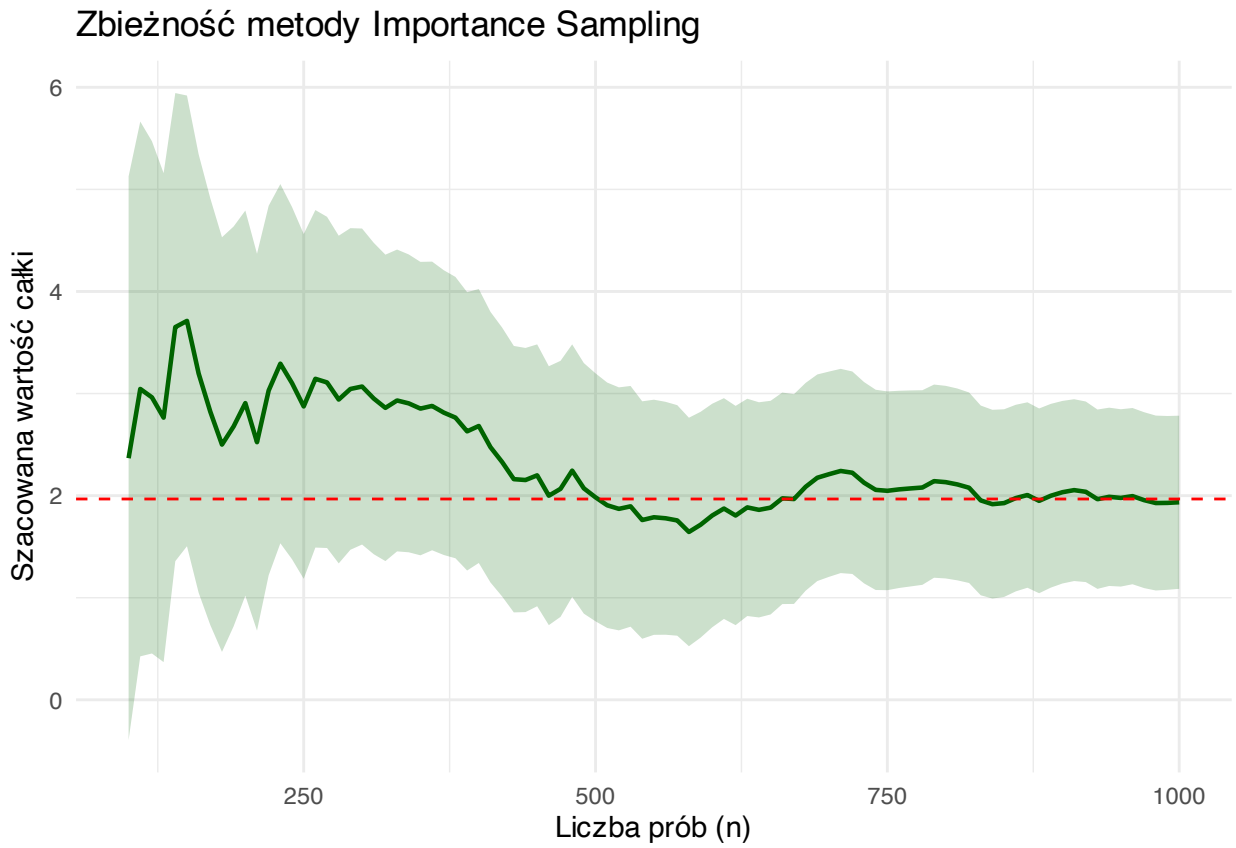
X_i rozkład jak powyżej.

Funkcja szacowania całki:

```
calculate_integral_importance <- function(n_samples = 50000) {

  f <- function(x) {
    val <- x * sin((1 / cos(log(x + 1)))^2)
    val[is.nan(val)] <- 0
    return(val)
  }


  u_rand <- runif(n_samples, min = 0, max = 1)

  x_samp <- 2 * pi * sqrt(u_rand)

  weight_factor <- 2 * pi^2
  h_values <- weight_factor * sin((1 / cos(log(x_samp + 1)))^2)

  cum_sum_h <- cumsum(h_values)
  cum_sum_sq <- cumsum(h_values^2)
  n_vec <- 1:n_samples

  est_integral <- cum_sum_h / n_vec

  mean_sq <- cum_sum_sq / n_vec
  mean_val <- est_integral

  sample_var <- (mean_sq - mean_val^2)

  se <- sqrt(sample_var / n_vec)

  z_score <- 1.96
  ci_lower <- est_integral - z_score * se
  ci_upper <- est_integral + z_score * se

  plot_indices <- seq(100, n_samples, by = 10)
  df_plot <- data.frame(
    n = plot_indices,
    value = est_integral[plot_indices],
    ci_l = ci_lower[plot_indices],
    ci_u = ci_upper[plot_indices]
  )

  final_result <- list(
    value = tail(est_integral, 1),
    ci_low = tail(ci_lower, 1),
    ci_high = tail(ci_upper, 1),
```

```
    data = df_plot
  )

  return(final_result)
}
```

A poniżej zastosowanie kodu na przykładzie z 95% przedziałem ufności:



Rysunek 5: Wykres szacowania wartości całki metodą importance sampling z przedziałami ufności

3 Zadanie 3: Obciążenie i metoda delta (2 pkt)

Załóżmy, że mamy wygenerowaną próbę prostą $(x_j)_{j=1}^n$ ze zmiennej X o rozkładzie normalnym $\mathcal{N}(0, \sigma^2)$, gdzie $\sigma^2 > 0$ jest nieznane. Chcemy estymować wartości funkcji tworzącej momenty (czyli transformaty Laplace'a)

$$M_X(t) = \mathbb{E}e^{tX}.$$

Rozważmy dwie metody:

1. Estymacja $\widehat{M_X}(t) = \mathbb{E}e^{tX}$ metodą Monte-Carlo na podstawie próby $(x_j)_{j=1}^n$.
2. Estymacja wariancji σ^2 za pomocą wariancji próbkowej σ_n^2 z $(x_j)_{j=1}^n$ i następnie obliczenie estymatora $\widehat{M_X}(t) = e^{\frac{t^2}{2}\sigma_n^2}$.

Oceń jakość obu metod estymacji, wyznaczając asymptotyczne przedziały ufności obu estymatorów dla rosnącej liczności próby n z rozkładu $\mathcal{N}(0, \sigma^2)$ i wybranego $\sigma^2 \neq 1$.

3.1 Rozwiązanie

3.1.1 zadanie 3

X ma rozkład $\mathcal{N}(0, \sigma^2)$, a wartość, którą chcemy estymować to

$$M_X(t) = \mathbb{E}e^{tX} = e^{\frac{\sigma^2 t^2}{2}}$$

Metoda 1

Definiujemy zmienną losową Y $= e^{tX}$. Nasz estymator to średnia z próby:

$$\hat{\theta}_1 = \frac{1}{n}\sum_{j=1}^{n} e^{tx_j}$$

Zgodnie z Centralnym Twierdzeniem Granicznym (CTG), estymator ten ma rozkład asymptotycznie normalny:

$$\sqrt{n}(\hat{\theta}_1 - M_X(t)) \xrightarrow{d} \mathcal{N}(0, \sigma_Y^2)$$

gdzie wariancja σ_Y^2 wynosi:

$$\sigma_Y^2 = Var(e^{tX}) = e^{2\sigma^2 t^2} - e^{\sigma^2 t^2}$$

Asymptotyczny przedział ufności (95%) dla Metody 1:

$$\left[\hat{\theta}_1 - 1.96\frac{\hat{S}_Y}{\sqrt{n}}, \hat{\theta}_1 + 1.96\frac{\hat{S}_Y}{\sqrt{n}}\right]$$

gdzie $\hat{S}_Y$ to odchylenie standardowe z próby wartości e^{tx_j}.

Metoda 2: Metoda Delta

Chcemy estymoawć σ^2 za pomocą $\hat{\sigma}_n^2 = \frac{1}{n}\sum_{j=1}^n x_j^2$ (estymator NW dla $\mu = 0$). Nasz estymator to:

$$\hat{\theta}_2 = g(\hat{\sigma}_n^2) = e^{\frac{t^2}{2}\hat{\sigma}_n^2}$$

Wiemy, że dla zmiennej normalnej X^2 ma wariancję $Var(X^2) = 2\sigma^4$. Zatem z CTG:

$$\sqrt{n}(\hat{\sigma}_n^2 - \sigma^2) \xrightarrow{d} \mathcal{N}(0, 2\sigma^4)$$

Stosujemy Metodę Delta ($g'(u) = \frac{t^2}{2} e^{\frac{t^2}{2}u}$). Wariancja asymptotyczna estymatora $\hat{\theta}_2$ wynosi:

$$Var(\hat{\theta}_2) \approx [g'(\sigma^2)]^2 \cdot Var(\hat{\sigma}_n^2) = \frac{1}{n} \cdot \frac{t^4\sigma^4}{2} \cdot e^{\sigma^2 t^2}$$

Następnie wyznaczamy asymtotyczny przedział ufności (95%):

$$\left[\ \hat{\theta}_2 - 1.96 \cdot SE, \hat{\theta}_2 + 1.96 \cdot SE\ \right]$$

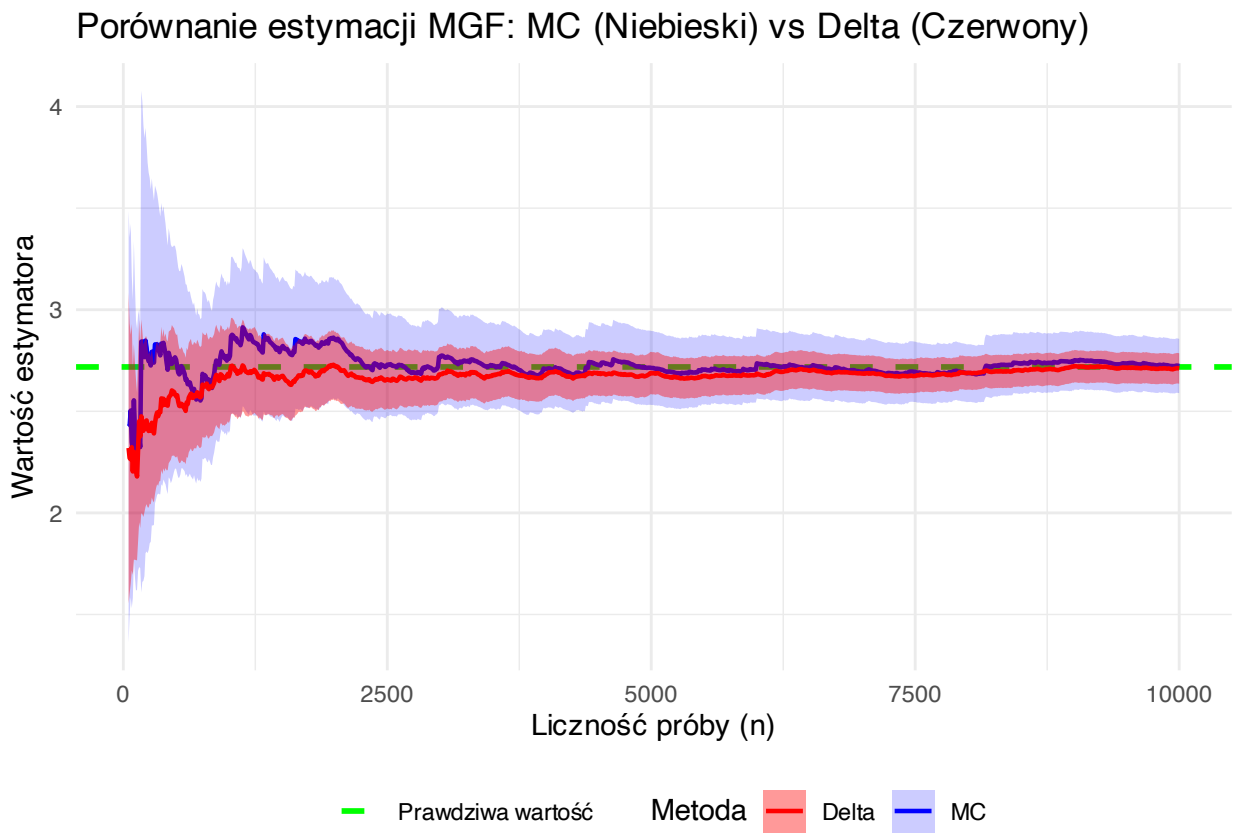
gdzie błąd standardowy $SE = \sqrt{\frac{1}{n} \cdot \frac{t^4(\sigma_n^2)^2}{2} \cdot e^{\sigma_n^2 t^2}}$

Poniżej znjaduje się kod, który oblicza obydwa estymatory:

```
compare_mgf_estimators <- function(N = 5000, sigma_sq = 2, t = 1) {

  true_value <- exp((sigma_sq * t^2) / 2)
  sigma <- sqrt(sigma_sq)

  x <- rnorm(N, mean = 0, sd = sigma)

  y <- exp(t * x)

  n_seq <- 1:N
  est_1 <- cumsum(y) / n_seq

  mean_y_sq <- cumsum(y^2) / n_seq
  var_1 <- (mean_y_sq - est_1^2)
  var_1[var_1 < 0] <- 0

  se_1 <- sqrt(var_1 / n_seq)

  ci_1_lower <- est_1 - 1.96 * se_1
  ci_1_upper <- est_1 + 1.96 * se_1

  x_sq <- x^2
  sigma_sq_hat <- cumsum(x_sq) / n_seq

  est_2 <- exp((t^2 / 2) * sigma_sq_hat)

  derivative <- (t^2 / 2) * est_2
  asymp_var_param <- 2 * (sigma_sq_hat)^2

  se_2 <- sqrt((derivative^2 * asymp_var_param) / n_seq)

  ci_2_lower <- est_2 - 1.96 * se_2
  ci_2_upper <- est_2 + 1.96 * se_2

  idx <- seq(50, N, by = 10)

  df_plot <- data.frame(
    n = idx,
```

```
    est_1 = est_1[idx],
    ci_1_l = ci_1_lower[idx],
    ci_1_u = ci_1_upper[idx],
    est_2 = est_2[idx],
    ci_2_l = ci_2_lower[idx],
    ci_2_u = ci_2_upper[idx]
  )

  return(list(data = df_plot, true_val = true_value, params = list(s2=sigma_sq, t=t)))
}
```

Zkolei tutaj już mamy porówananie wizualne tych estymatrów



Rysunek 6: Wykres porównania działania estymatorów (Metoda 1 vs Metoda 2)

Jak widać Metod delta ma znacznie węższy przedział ufności.

4 Zadanie 4: Zmniejszanie wariancji w estymacji całek metodami Monte-Carlo (4 pkt)

1. Zaproponuj przynajmniej trzy różne estymatory Monte-Carlo estymujące wartość całki

$$I_1 = \int_{\frac{1}{2}}^{1} \frac{e^{1/x} x^{1/3}}{(1+x)^{7/3}} \mathrm{d}x.$$

Skorzystaj z różnych metod zaprezentowanych na wykładzie. Porównaj zaproponowane estymatory, analizując ich wariancje bądź asymptotyczne przedziały ufności. Określ, który z estymatorów jest najlepszy.

2. Zaproponuj estymator Monte-Carlo całki

$$I_2 = \int_{\frac{1}{2}}^{\infty} \frac{e^{1/x} x^{1/3}}{(1+x)^{7/3}} \mathrm{d}x.$$

Uwaga 1: Ogłaszam konkurs na najlepszy estymator całki I_2. Osoba, która zaproponuje najefektywniejszy (o najwęższym asymptotycznym przedziale ufności - czyli jednocześnie najmniejszej wariancji), będzie darzona nieskończoną **estymą** i dodatkowo otrzyma 5 punktów z aktywności (czyli maksymalną możliwą liczbę punktów).

Uwaga 2: Chociaż w funkcji podcałkowej występuje gęstość **odwróconego rozkładu Beta** z parametrami $\alpha = \frac{4}{3}$ i $\beta = 1$, to estymator Monte-Carlo oparty o próbę generowaną z tego rozkładu jest wyjątkowo nieefektywny (można się zastanowić dlaczego). Można jednak poczytać w jaki sposób taki odwrócony rozkład Beta się otrzymuje i w oparciu o tę wiedzę spróbować wymyślić bardziej efektywny estymator.

4.1 Rozwiązanie

4.1.1 Zadanie 4.1

Zadaniem jest estymacja całki:

$$I_1 = \int_{\frac{1}{2}}^{1} \frac{e^{1/x} x^{1/3}}{(1+x)^{7/3}} \mathrm{d}x = \int_{\frac{1}{2}}^{1} f(x) dx.$$

Do tego zadania wybieram 4 różne estymatory MC i w poniższej części opiszę je po kolei:

1. Zwykły estymator Monte Carlo

Losuje punkty z rozkładu jednostajnego na przedziale [0.5,1], obliczam średnią wartość funkcji i mnożę przez długość przedziału (L=0.5). Estymator:

$$\hat{I}_{MC} = L \cdot \frac{1}{n} \sum_{i=1}^{n} f(U_i)$$

2. Zmienne Antytetyczne

Wykorzystuje fakt, że funkcja $f(x)$ jest monotoniczna (malejąca). Jeśli wylosuje U z rozkładu jednostajnego, to U i (1−U) są ujemnie skorelowane. W przedziale $[0.5, 1]$, parą antytetyczną dla x będzie $x' = 1.5 - x$. Estymator:

$$\hat{I}_{AV} = L \cdot \frac{1}{n} \sum_{i=1}^{n} \frac{f(x_i) + f(1.5 - x_i)}{2}$$

3. Importance Sampling

Chcemy losować więcej punktów tam, gdzie funkcja przyjmuje duże wartości (blisko x=0.5). Zastosujemy gęstość próbkującą liniową malejącą: $h(x) = 8(1-x)$ na przedziale $[0.5, 1]$. Dystrybuanta odwrotna do generowania próby: $X = 1 - 0.5\sqrt{U}$, gdzie $U \sim \mathcal{U}(0,1)$. Waga $w(x) = \frac{1}{h(x)}$. Estyamtor:

$$\hat{I}_{IS} = \frac{1}{n}\sum_{i=1}^{n} f(X_i)w(X_i)$$

4. Rao - Blackwell

Zauważmy, że funkcję podcałkową możemy zapisać w postaci związanej z gęstością prawdopodobieństwa. Niech X będzie zmienną losową o rozkładzie odwróconego Beta z parametrami $\alpha = 4/3$ oraz $\beta = 1$. Jej gęstość wyraża się wzorem:

$$g(x) = \frac{1}{B(4/3,1)}\frac{x^{4/3-1}}{(1+x)^{4/3+1}} = \frac{4}{3}\frac{x^{1/3}}{(1+x)^{7/3}},\ x > 0$$

Możemy więc wartość całki I zapisać jako wartość oczekiwną:

$$I_1 = \mathbb{E}\left[\frac{3}{4}e^{1/X}\mathbf{1}_{[0.5,1]}(X)\right]$$

Zauważmy jednak, że jeżeli mamy dwie niezależne zmienne losowe $U \sim \Gamma(4/3,1)$ oraz $U \sim \Gamma(1,1)$ $(= \mathrm{Exp}(1))$ wówczas $X = \frac{U}{V}$ ma rozkład odwrotny Beta (4/3,1). Całkę można zapisać w postaci:

$$\mathbb{E}\left[\frac{3}{4}e^{V/U}\,\mathbf{1}_{\{0.5\le\frac{U}{V}\le 1\}}\right] = \mathbb{E}_U\left(\mathbb{E}\left[\frac{3}{4}e^{V/U}\,\mathbf{1}_{\{U\le V\le 2U\}}\,\middle|\,U=u\right]\right).$$

Obliczam warunkową wartość oczekiwaną (całkując po rozkładzie V, którego gęstość to e^{-v}):

$$\tilde{h}(u) = \mathbb{E}[\cdots|U=u] = \frac{3}{4}\int_u^{2u} e^{v(\frac{1}{u}-1)}dv$$

Dla u $\neq$ 1:

$$\tilde{h}(u) = \frac{3}{4}\int_u^{2u} e^{v(\frac{1}{u}-1)}dv = \frac{3}{4}\frac{u}{1-u}(e^{2(1-u)} - e^{1-u})$$

Dla u = 1:

$$\tilde{h}(1) = \frac{3}{4}\int_1^{2} 1dv = \frac{3}{4}$$

Możemy więc zaproponować estymator Rao-Blackwella postaci:

$$\hat{I}_{RB} = \frac{1}{n}\sum_{i=1}^{n}\tilde{h}(U_i)$$

Gdzie U_i ą niezależnymi zmiennymi losowymi z. rozkładu $\Gamma(4/3,1)$

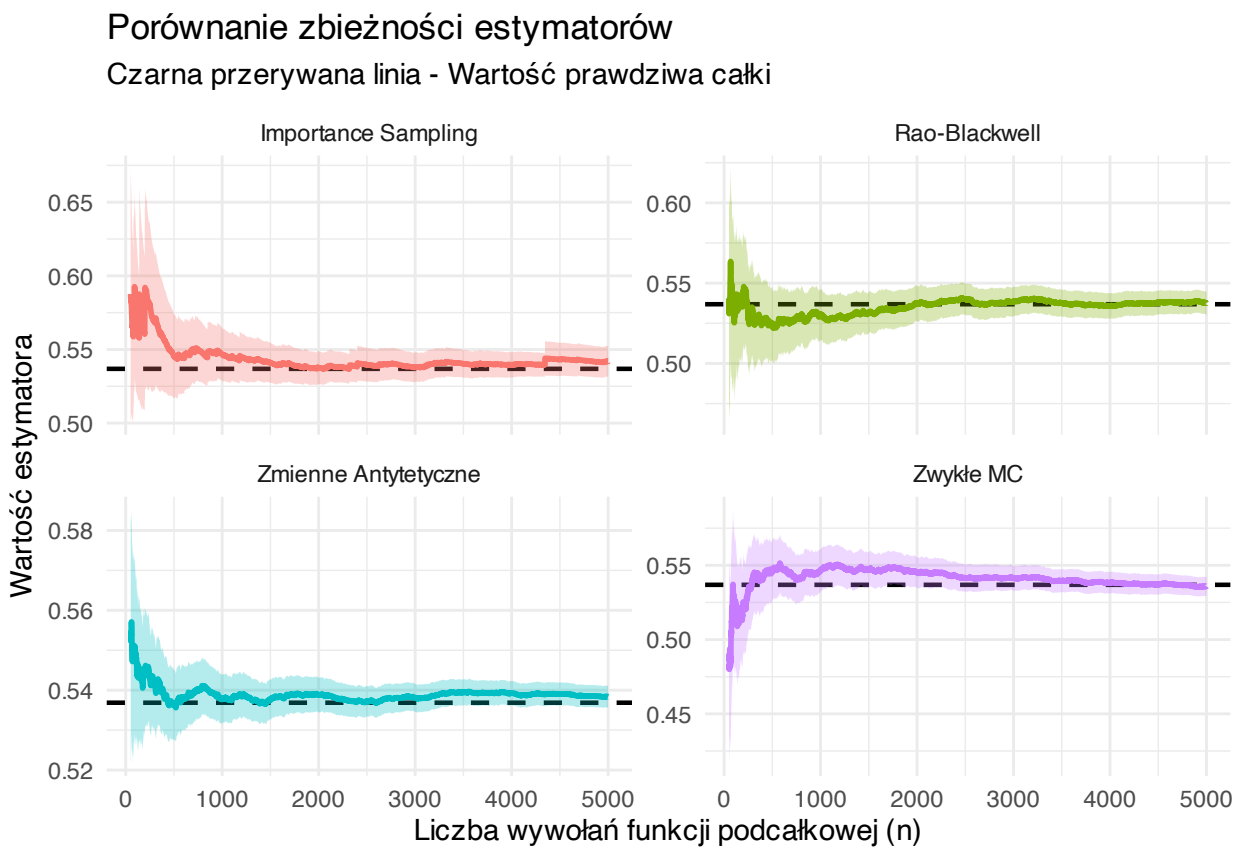
Poniżej znajduje się kod na podstawie obliczeń:

```
f <- function(x) {
  (exp(1/x) * x^(1/3)) / (1+x)^(7/3)
}

true_val_integration <- integrate(f, 0.5, 1)$value
```

```r
generate_plot_data <- function(N = 5000) {

  u_cmc <- runif(N, 0.5, 1)
  y_cmc <- f(u_cmc) * 0.5

  n_seq <- 1:N
  est_cmc <- cumsum(y_cmc) / n_seq
  est_sq_cmc <- cumsum(y_cmc^2) / n_seq
  var_cmc <- (est_sq_cmc - est_cmc^2) / n_seq
  sd_cmc <- sqrt(pmax(var_cmc, 0))

  df_cmc <- data.frame(
    n = n_seq,
    est = est_cmc,
    ci_l = est_cmc - 1.96 * sd_cmc,
    ci_u = est_cmc + 1.96 * sd_cmc,
    Metoda = "Zwykłe MC"
  )

  N_pairs <- floor(N / 2)
  u_av <- runif(N_pairs, 0.5, 1)
  u_ant <- 1.5 - u_av

  y_av <- (f(u_av) * 0.5 + f(u_ant) * 0.5) / 2

  n_seq_av <- 1:N_pairs
  est_av <- cumsum(y_av) / n_seq_av
  est_sq_av <- cumsum(y_av^2) / n_seq_av
  var_av <- (est_sq_av - est_av^2) / n_seq_av
  sd_av <- sqrt(pmax(var_av, 0))

  df_av <- data.frame(
    n = n_seq_av * 2,
    est = est_av,
    ci_l = est_av - 1.96 * sd_av,
    ci_u = est_av + 1.96 * sd_av,
    Metoda = "Zmienne Antytetyczne"
  )

  u_is <- runif(N)
  x_is <- 1 - 0.5 * sqrt(u_is)
  weights <- 1 / (8 * (1 - x_is))
  y_is <- f(x_is) * weights

  est_is_cum <- cumsum(y_is) / n_seq
  est_sq_is <- cumsum(y_is^2) / n_seq
  var_is <- (est_sq_is - est_is_cum^2) / n_seq
  sd_is <- sqrt(pmax(var_is, 0))

  df_is <- data.frame(
    n = n_seq,
    est = est_is_cum,
    ci_l = est_is_cum - 1.96 * sd_is,
```

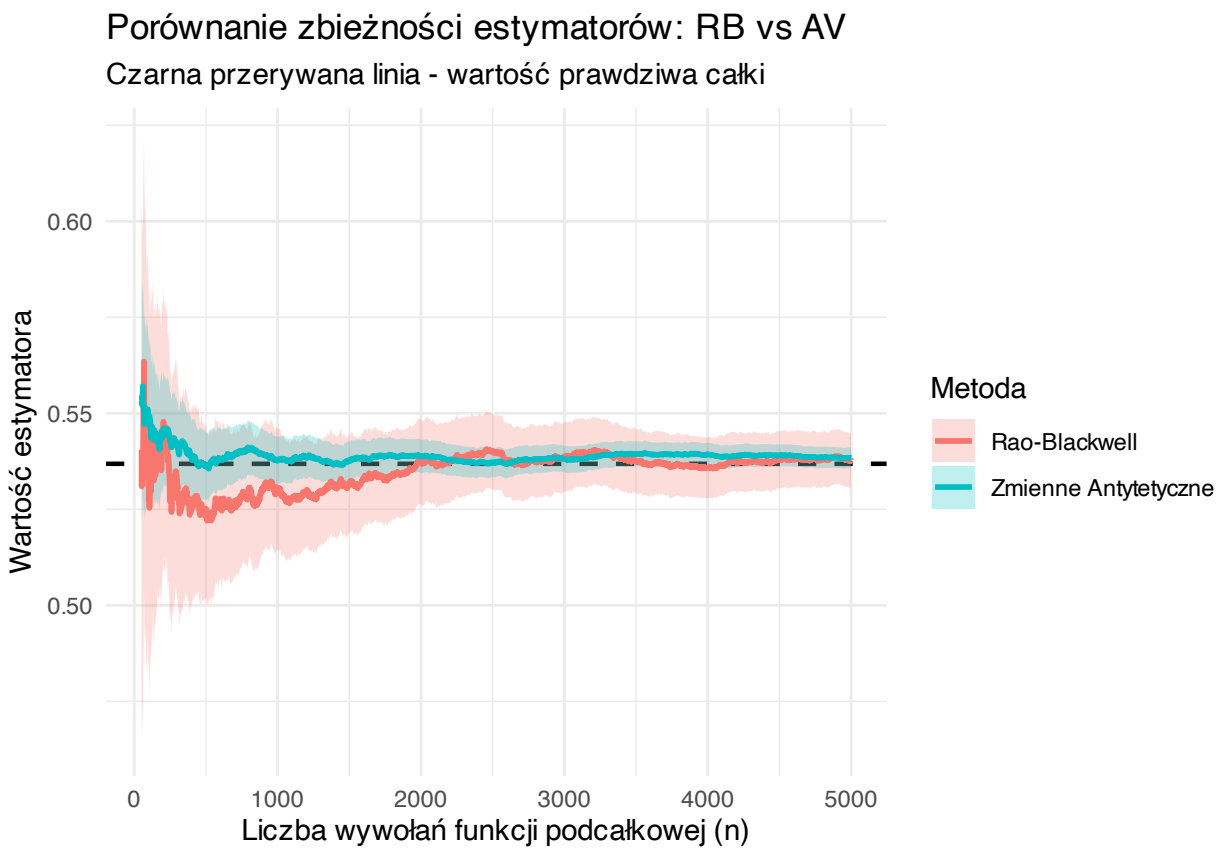
```r
    ci_u = est_is_cum + 1.96 * sd_is,
    Metoda = "Importance Sampling"
  )

  h_tilde_func <- function(u) {
    z <- 1 - u
    out <- numeric(length(u))
    close <- abs(z) < 1e-10
    out[!close] <- (3/4) * u[!close] * exp(z[!close]) * expm1(z[!close]) / z[!close]

    out[close] <- 0.75

    return(out)
  }

  u_rb_sample <- rgamma(N, shape = 4/3, rate = 1)
  y_rb <- h_tilde_func(u_rb_sample)

  est_rb <- cumsum(y_rb) / n_seq
  est_sq_rb <- cumsum(y_rb^2) / n_seq
  var_rb <- (est_sq_rb - est_rb^2) / n_seq
  sd_rb <- sqrt(pmax(var_rb, 0))

  df_rb <- data.frame(
    n = n_seq,
    est = est_rb,
    ci_l = est_rb - 1.96 * sd_rb,
    ci_u = est_rb + 1.96 * sd_rb,
    Metoda = "Rao-Blackwell"
  )

  final_df <- rbind(df_cmc, df_av, df_is, df_rb)
  return(final_df)
}
```

Poniżej znajdują się wykresy porównujące estymatory



Rysunek 7: Wykresy estymatorów MC, AV, IS, RB

Jak widać z powyższych wykresów najwęższymi przedziałami ufności cechują się estymatory Rao-Blackwella i zmiennych antytetycznych, więc te dwie metody porównam na osobnym wykresie, aby sprawdzić, która jest lepsza.



Rysunek 8: Wykres porównujący estymatory AV i RB

Zauważalnie lepszy wydaje się być estymator polegający na zmiennych antytytetycznych, ponieważ jego przedział ufności, jak widać na rysunku znacznie rzadziej jest większych rozmiarów niż przedział ufności estymatora Rao-Blackwella.

4.1.2 Zadanie 4.2

Tym razem chcę estymować poniższą całkę wszelkimi sposobami tak aby jej przedział ufności był jak najwęższy:

$$I_2 = \int_{\frac{1}{2}}^{\infty} \frac{e^{1/x} x^{1/3}}{(1+x)^{7/3}} \mathrm{d}x.$$

Sposób, który znalazłem w internecie prawie na pewno daje najmniejszą możliwą wariancję. Jest dość zaawansowany matematycznie i ze względu na jedną część nie wiem, czy jest dozwolony w konkursie ale, po angielsku ten sposób nazywa się **Taylor Expansion Control Variate**.

Na początku zastosuję podstawienie, które umożliwi mi zastosowanie wspomnianej metody:

$$x = \frac{y}{1-y} \implies y = \frac{x}{x+1}, \quad dx = \frac{1}{(1-y)^2} dy$$

Granice całki zmienią się wtedy następująco $x = 1/2 \implies y = 1/3$ oraz $x \to \infty \implies y \to 1$, teraz podstawiając to wszytsko do całki mamy:

$$I_2 = \int_{\frac{1}{3}}^{1} e^{\frac{1-y}{y}} y^{1/3} \mathrm{d}y.$$

Dla wygody będę oznaczał wykładnik $\frac{1-y}{y}$ literą T. Czynnik e^T jest głównym źródłem wariancji w całce i zastępuje go szeregiem Taylora tak aby był całkowalny analitycznie. Rozwijając funckję e^T wokół zera wychodzi nam, że:

$$e^T \approx \sum_{k=0}^{m} \frac{T^k}{k!},$$

gdzie m to rząd przybliżenia (im większe m tym mniejsza wariancja).

Zmienną kontrolną oznaczam h(y) i wygląda ona następująco:

$$h(y) = \Big(\sum_{k=0}^{m} \frac{T^k}{k!} \Big) \cdot y^{1/3}.$$

Teraz rozbijam całkę na dwie części: pierwsza liczona analitycznie, druga metodą montecarlo:

$$I_2 = \int_{\frac{1}{3}}^{1} h(y)\mathrm{d}y \ + \ \int_{\frac{1}{3}}^{1} [e^{T(y)} y^{1/3} - h(y)]\mathrm{d}y$$

Część Analityczna:

Całkę z h(y) dzięki liniowości można obliczyć "wyraz po wyrazie":

$$\int_{\frac{1}{3}}^{1} h(y)\mathrm{d}y = \sum_{k=0}^{m} \frac{1}{k!} \int_{\frac{1}{3}}^{1} T(y)^k y^{1/3} \mathrm{d}y = \sum_{k=0}^{m} \frac{1}{k!} \int_{\frac{1}{3}}^{1} (1-y)^k y^{(1/3)-k} \mathrm{d}y$$

Wartości tej całki w kodzie liczy funkcja *I_k_exact* (Nie wiem czy taki ruch jest dozwolony w konkursie)

Część Monte Carlo:

Część tą zapiszę wyciągając przed nawias $y^{1/3}$ jak poniżej:

$$R = \int_{\frac{1}{3}}^{1} \Big(e^{T(y)} - \sum_{k=0}^{m} \frac{T^k}{k!} \Big) y^{1/3} \mathrm{d}y$$

Estymuje tę całkę metodą Monte Carlo na przedziale [1/3,1]. Długość przedziału to V = 2/3, więc:

$$\hat{R}_{MC} = \frac{2}{3} \cdot \frac{1}{N} \cdot \sum_{i=1}^{N} \Bigg[\Big(e^{T(U_i)} - \sum_{k=0}^{m} \frac{T(U_i)^k}{k!} \Big) U_i^{1/3} \Bigg]$$

gdzie oczywiście $U_i \sim \mathcal{U}[1/3, 1]$.

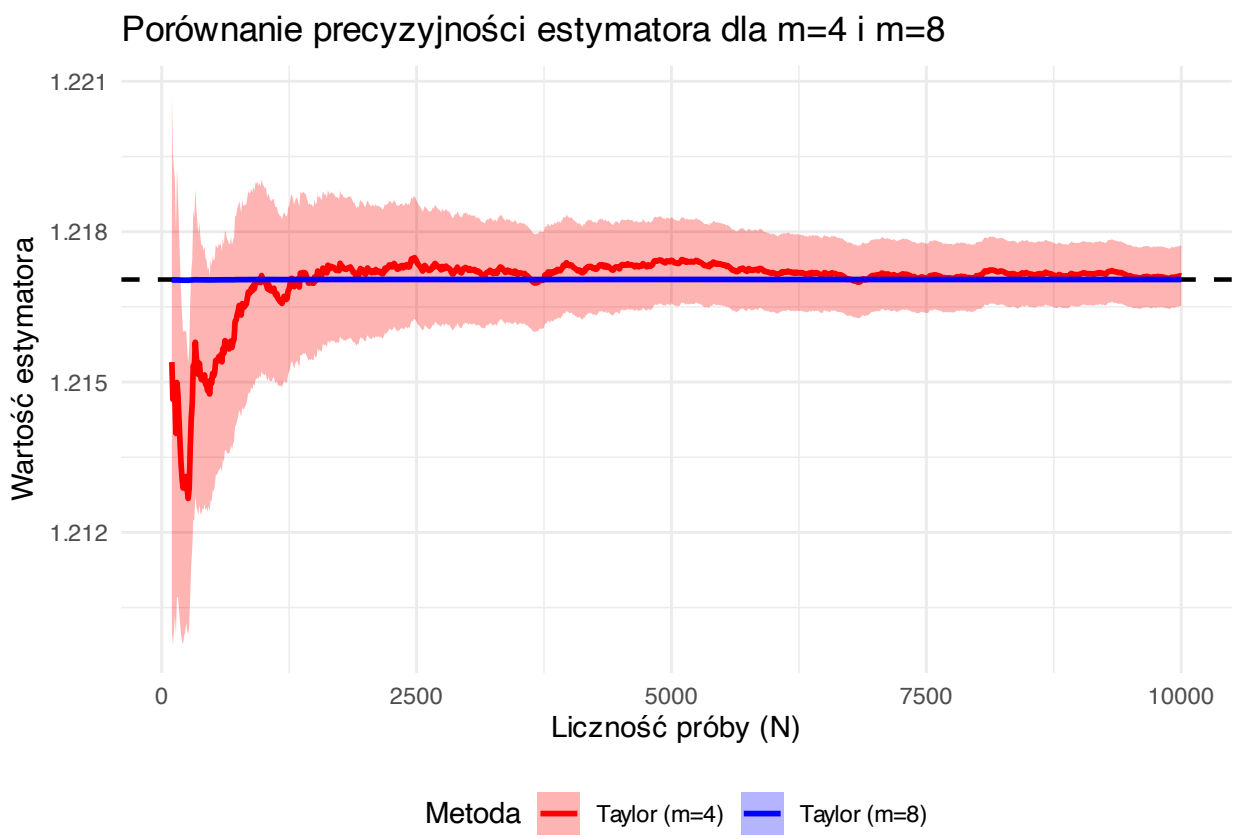
Ostateczny Wzór Estymatora:

$$\hat{I}_{Taylor} = \sum_{k=0}^{m} \frac{1}{k!} \int_{\frac{1}{3}}^{1} (1-y)^k y^{(1/3)-k} \mathrm{d}y \ + \ \frac{2}{3} \cdot \frac{1}{N} \cdot \sum_{i=1}^{N} \Bigg[\Big(e^{T(U_i)} - \sum_{k=0}^{m} \frac{T(U_i)^k}{k!} \Big) U_i^{1/3} \Bigg]$$

Kod do obliczeń:

```r
I_k_exact <- function(k, a = 1/3, b = 1) {
  j <- 0:k
  coeff <- choose(k, j) * (-1)^j
  p <- 4/3 - k + j
  sum(coeff * (b^p - a^p) / p)
}

const_exact_taylor <- function(m, a = 1/3, b = 1) {
  sum(sapply(0:m, function(k) I_k_exact(k, a, b) / factorial(k)))
}

taylor_poly_exp <- function(t_val, m) {
  res <- rep(1, length(t_val))
  term <- rep(1, length(t_val))
  if (m > 0) {
    for (k in 1:m) {
      term <- term * t_val / k
      res <- res + term
    }
  }
  res
}

calc_taylor_estimator <- function(m, N, U, T_val, a = 1/3, b = 1) {
  stopifnot(length(U) == N, length(T_val) == N)

  Z0 <- b - a
  c0 <- const_exact_taylor(m, a, b)

  Pm <- taylor_poly_exp(T_val, m)
  r <- (exp(T_val) - Pm) * U^(1/3)

  n <- seq_len(N)
  cum_mean <- cumsum(r) / n

  cum_sum2 <- cumsum(r^2)

  s2 <- rep(NA_real_, N)
  if (N >= 2) {
    s2[2:N] <- (cum_sum2[2:N] - n[2:N] * cum_mean[2:N]^2) / (n[2:N] - 1)
  }

  sd_vals <- rep(NA_real_, N)
  if (N >= 2) {
    sd_vals[2:N] <- Z0 * sqrt(pmax(s2[2:N], 0)) / sqrt(n[2:N])
  }

  est_vals <- c0 + Z0 * cum_mean

  list(
    est = est_vals,
    sd = sd_vals,
    final_est = est_vals[N],
```

```
    final_sd = sd_vals[N]
  )
}
```



Rysunek 9: Porówananie estymatora dla m=4 i m=8

Tabela 5: Porównanie efektywności estymatorów po N = 10000 symulacjach

Metoda	Estymata	Błąd Standardowy	Szerokość CI 95%
Taylor (m=4)	1.217116	0.000306690392	0.001202226335
Taylor (m=8)	1.217046	0.000001080175	0.000004234286

Jak widać dla coraz wyższych m asymptotyczny przedział ufności się zawęża, można nawet przypuszczać, że dla $m \to \infty$ ten przedział będzie "nieskończenie wąski", jeśli można to tak określić.